第38课 动态规划入门（斐波那契、爬楼梯）

本课为选学内容，听不懂没关系，以后还会再学！

> 本讲义配合课堂例题讲解使用，代码可直接在 Dev-C++ 中运行验证。

五、课堂例题

例题 1：斐波那契数列——递归版 vs DP 版

题目：输入正整数 n（1 ≤ n ≤ 100），输出斐波那契数列的第 n 项。

讲解思路：

1. 先回顾递归版——让学生体验 n=40 时的龟速。
2. 引出 DP 版——用数组从小到大存每一项。
3. 让学生对比两个程序的运行速度，感受 DP 的威力。

递归版（慢，只做对比用）：

```
#include <iostream>
using namespace std;

int fib(int n) {
    if (n == 1 || n == 2) return 1;
    return fib(n - 1) + fib(n - 2);
}

int main() {
    int n;
    cin >> n;
    cout << fib(n) << endl;
    return 0;
}
```

> 试试输入 40——你会发现电脑要想好久！输入 50？可能直接卡死！

DP 版（快！）：

```
#include <iostream>
using namespace std;

long long dp[105];  // 用数组存每一项的答案（备忘录！）

int main() {
    int n;
    cin >> n;

    // 第一步：初始化最小的问题（起点）
    dp[1] = 1;
    dp[2] = 1;

    // 第二步：从小到大，一步步推出每一项
    for (int i = 3; i <= n; i++) {
        dp[i] = dp[i - 1] + dp[i - 2];  // 状态转移方程！
    }

    // 第三步：直接输出答案
    cout << dp[n] << endl;

    return 0;
}
```

输入输出示例：

```
输入：10
输出：55

输入：50
输出：12586269025
```

> DP 版算 fib(50) 都是瞬间出结果！而递归版卡到天荒地老。

板书：为什么快？

```
递归版：fib(5) 要计算 9 次函数调用（很多重复）
DP 版：只需要一个 for 循环跑 3 次（i=3,4,5），零重复！

递归版算 fib(40)：约 3 亿次计算  好几秒
DP 版算 fib(40)：38 次计算  一瞬间
```

课堂互动：让学生同时运行两个程序，输入 40，感受天壤之别。

例题 2：爬楼梯

题目：一共有 n 级台阶（1 ≤ n ≤ 50），每次可以走 1 级或 2 级，请问走到第 n 级共有多少种不同的走法？

讲解思路：

1. 引导学生发现：到第 i 级只能从第 i-1 级（走1步）或第 i-2 级（走2步）过来。
2. 状态转移方程：`dp[i] = dp[i-1] + dp[i-2]`。
3. 初始值：`dp[1] = 1, dp[2] = 2`。
4. 注意：这里 `dp[2] = 2`（两种走法：1+1 或 2），和斐波那契的 `dp[2] = 1` 不同！

```
#include <iostream>
using namespace std;

long long dp[55];  // 存每一级台阶的走法数

int main() {
    int n;
    cin >> n;

    dp[1] = 1;  // 第1级：只有1种走法（走1步）
    dp[2] = 2;  // 第2级：2种走法（1+1 或 2）

    for (int i = 3; i <= n; i++) {
        dp[i] = dp[i - 1] + dp[i - 2];  // 从前两级转移过来
    }

    cout << dp[n] << endl;

    return 0;
}
```

输入输出示例：

```
输入：5
输出：8

输入：10
输出：89
```

板书：手动填表验证

```
i:     1   2   3   4   5   6   7   8   9  10
dp[i]: 1   2   3   5   8  13  21  34  55  89

dp[3] = dp[2] + dp[1] = 2 + 1 = 3 （三种走法：1+1+1, 1+2, 2+1）
dp[4] = dp[3] + dp[2] = 3 + 2 = 5
dp[5] = dp[4] + dp[3] = 5 + 3 = 8
```

> 引导学生观察：爬楼梯的答案序列 1, 2, 3, 5, 8, 13, 21 其实就是斐波那契数列往后挪了一位！

例题 3（拓展）：一次可以走 1、2 或 3 级

题目：一共有 n 级台阶（1 ≤ n ≤ 50），每次可以走 1 级、2 级或 3 级，请问走到第 n 级共有多少种不同的走法？

讲解思路：

1. 和例题 2 类似，但多了一种选择——从第 i-3 级跨 3 步上来。
2. 状态转移方程变成：`dp[i] = dp[i-1] + dp[i-2] + dp[i-3]`。
3. 需要三个初始值：`dp[1] = 1, dp[2] = 2, dp[3] = 4`。

```cpp
#include <iostream>
using namespace std;

long long dp[55];

int main() {
    int n;
    cin >> n;

    dp[1] = 1;  // 1种: (1)
    dp[2] = 2;  // 2种: (1,1) (2)
    dp[3] = 4;  // 4种: (1,1,1) (1,2) (2,1) (3)

    for (int i = 4; i <= n; i++) {
        dp[i] = dp[i - 1] + dp[i - 2] + dp[i - 3];
    }

    cout << dp[n] << endl;

    return 0;
}
```

输入输出示例：

```
输入：5
输出：13

输入：4
输出：7
```

板书：填表

```
i:      1   2   3   4    5    6
dp[i]: 1   2   4   7   13   24

dp[4] = dp[3] + dp[2] + dp[1] = 4 + 2 + 1 = 7
dp[5] = dp[4] + dp[3] + dp[2] = 7 + 4 + 2 = 13
```

> 这道题展示了 DP 的灵活性——状态转移方程变一下，就能解决新问题！以后遇到更复杂的 DP 题，思路也是一样的：先找"我从哪些小状态过来"，写出方程，然后从小到大填表。

附：例题源代码与运行结果

例题 2：Climbing Stairs

```
// There are n stairs. You can climb 1 or 2 steps at a time.
// How many distinct ways to reach the n-th stair?
#include <iostream>
using namespace std;

long long dp[55];

int main() {
    int n;
    cin >> n;

    dp[1] = 1;  // 1 stair: 1 way (take 1 step)
    dp[2] = 2;  // 2 stairs: 2 ways (1+1 or 2)

    for (int i = 3; i <= n; i++) {
        dp[i] = dp[i - 1] + dp[i - 2];  // Come from (i-1) or (i-2)
    }

    cout << dp[n] << endl;

    return 0;
}
```

Sample Input

```
5
```

Expected Output

```
8
```

例题 3：Climbing Stairs — 1, 2, or 3 Steps

```
// There are n stairs. You can climb 1, 2, or 3 steps at a time.
// How many distinct ways to reach the n-th stair?
#include <iostream>
using namespace std;

long long dp[55];

int main() {
    int n;
    cin >> n;

    dp[1] = 1;  // 1 way:  (1)
    dp[2] = 2;  // 2 ways: (1,1) (2)
    dp[3] = 4;  // 4 ways: (1,1,1) (1,2) (2,1) (3)

    for (int i = 4; i <= n; i++) {
        dp[i] = dp[i - 1] + dp[i - 2] + dp[i - 3];
    }

    cout << dp[n] << endl;

    return 0;
}
```

Sample Input

```
5
```

Expected Output

```
13
```